



运筹学实验手册

运筹学教学组

西北工业大学 管理学院

2025 年 3 月

目 录

1. 线性规划.....	1
1.1 实验目的.....	1
1.2 生产计划问题.....	1
1.2.1 数学模型的建立.....	1
1.2.2 线性规划的 OPL 模型.....	2
1.2.3 案例 1.....	2
1.3 数据包络分析.....	3
1.3.1 数学模型的建立.....	3
1.3.2 DEA 的 OPL 模型.....	4
1.3.3 案例 2.....	5
1.4 练习.....	6
2. 运输问题.....	8
2.1 实验目的.....	8
2.2 产销平衡运输问题.....	8
2.2.1 数学模型的建立.....	8
2.2.2 运输问题的 OPL 模型.....	9
2.2.3 案例 1.....	10
2.3 产销不平衡运输问题.....	11
2.3.1 供过于求.....	11
2.3.2 供不应求.....	11
2.3.3 案例 2.....	12
2.4 练习.....	13
3. 整数规划.....	15
3.1 实验目的.....	15
3.2 0-1 整数规划模型.....	15
3.3 仓库选址问题.....	15
3.3.1 数学模型的建立.....	16
3.3.2 仓库选址问题的 OPL 模型.....	16
3.3.3 案例 1.....	17
3.4 指派问题.....	18
3.4.1 数学模型的建立.....	19

3.4.2 指派问题的 OPL 模型	19
3.4.3 案例 2	20
3.5 练习	20
4. 图与网络分析	22
4.1 实验目的	22
4.2 最短路问题	22
4.2.1 数学模型的建立	22
4.2.2 最短路问题的 OPL 模型	23
4.2.3 案例 1	24
4.3 最大流问题	26
4.3.1 数学模型的建立	26
4.3.2 最大流问题的 OPL 模型	27
4.3.3 案例 2	28
4.4 练习	28

1.实验要求

【知识方面的要求】

初步掌握数学规划问题的数学优化模型的建模方法以及求解方法。

【实验方面的要求】

每名同学根据老师所给的题目，独立完成所选题目的数学规划模型建模和软件求解等全部实验工作。

每名同学首先要提交所建立的数学规划模型，经任课老师审阅通过后，再使用优化软件进行求解的操作。优化软件求解出结果后，要经任课老师现场演示验收，通过后再撰写实验报告。

实验结束后，每名同学要提交纸版的实验报告供学院留存。实验报告应该至少包括实验题目、同学建立的数学规划模型、使用优化软件的相应设置情况、使用优化软件对数学规划模型的求解结果、对实验题目所提问题的回答等部分内容。

【实验条件的要求】

实验所用电脑可以使用实验室的电脑，也可使用学生自己的电脑。优化软件使用 IBM ILOG OPL 优化建模语言。

【其他方面的要求】

同学要独立完成实验的全部工作，不得抄袭。

2.实验步骤和内容

【学习优化软件】

任课教师为学生讲解 IBM ILOG OPL 优化建模语言的基本功能和操作方法。其他优化软件的使用方法可以自学掌握。

【建立数学规划模型】

建立实验题目的数学规划模型，并上交任课教师审核，审核通过后可以进入下一步。

【求解数学规划模型】

使用优化软件 CPLEX 求解 3.2 步建立的数学规划模型，并为任课教师现场演示求解过程。

【撰写提交实验报告】

同学撰写并提交实验报告。

3.实验成果及成绩评定

【实验成果描述】

建立的数学规划模型要正确、精炼。

优化软件求解过程要清晰，求解结果正确。

实验报告包含内容全面，论述语言简练准确，格式符合规范。

【实验成绩的分档及评定】

建立的数学规划模型部分成绩占实验成绩的 40%，优化软件求解部分成绩占实验成绩的 50%，实验报告部分成绩占实验成绩的 10%。

1. 线性规划

线性规划是一种对问题进行求解的方法，可以帮助管理者制定决策。以下是几种应该使用线性规划方法的典型情况：

(1) 制造者希望建立一个生产时间表和库存计划，以满足未来一段时间的市场需求。最理想的情况是，既满足市场上产品的需求，同时又使生产和库存的成本最低。

(2) 金融分析员必须选择一种股票或证券进行投资。金融分析员希望是自己的投资有最大的回报率。

(3) 营销经理希望能够从广播、电视、报纸、杂志这几种媒体中选择一种合适的组合，确定广告预算是自己的广告效益最好。

(4) 公司的仓库分布于全美各地，现在有一些顾客订单，公司希望确定每个仓库的发货量，使总成本最低。

这些只是线性规划应用的一小部分，但是可以看出线性规划广泛的应用。经过仔细的观察，我们可以发现这些例子有共同的特点。每个例子都要求我们使用实现目标函数值最大化或最小化。另外，存在约束条件，而且这些约束条件会影响目标的实现。

1.1 实验目的

- (1) 理解线性规划的概念。
- (2) 对于一个问题，能够建立基本的线性规划模型。
- (3) 学会运用 OPL 语言解决线性规划问题。
- (4) 将不同问题转化为线性规划的数学模型。

1.2 生产计划问题

1.2.1 数学模型的建立

给出 m 种资源 $r_1, r_2, \dots, r_m$ 的上限 $b_1, b_2, \dots, b_m$ ，用于生产 n 种产品 $p_1, p_2, \dots, p_n$ 各 $x_1, x_2, \dots, x_n$ 件。给出每单位产品的利润 $c_1, c_2, \dots, c_n$ 及单位 p_j 产品消耗 r_i 资源的量为 a_{ij} ，要求制定生产计划，使利润最大。

- (1) 数据的组织。这类问题的已经数据可以组织成向量和矩阵的形式：

$$b = (b_1, b_2, \dots, b_m), \quad c = (c_1, c_2, \dots, c_n), \quad a = \begin{pmatrix} a_{11}, \dots, a_{1n} \\ \vdots \\ a_{m1}, \dots, a_{mn} \end{pmatrix}$$

- (2) 数学模型。设： x 代表决策变量向量。

$$x = (x_1, x_2, \dots, x_n)^T$$

因此，数学模型可以写成紧凑形式，如下所示：

$$\begin{aligned} \max z &= \sum_{j=1}^n c_j \cdot x_j \\ \text{s. t. } &\begin{cases} \sum_{j=1}^n a_{ij} \cdot x_j \leq b_i & (i=1, 2, \dots, m) \\ x_j \geq 0 & (j=1, 2, \dots, n) \end{cases} \end{aligned}$$

1.2.2 线性规划的 OPL 模型

(1) 数据的定义及初始化。

定义集合 **Products** 为所有产品的集合，集合中所有的元素做数组的下标，数组 **c[Products]** 为每种产品的单位利润；定义 **Resources** 为所有资源的集合，集合中所有元素做数组的下标，**b[Resources]** 为每种资源的数量；二维数组 **a[Resources][Products]** 为生产每种单位产品消耗资源的数量。

```
{string} Products=...;
{string} Resources=...;
float b[Resources]=...;
float a[Resources][Products]=...;
float c[Products]=...;
```

(2) 决策变量的定义。定义 **x[Products]** 为每种产品的产量。

```
dvar float+ x[Products];
```

(3) 目标函数表达式。按照决策变量所给出的生产计划进行生产所产生的利润的最大化。

```
maximize sum (p in Products ) c[p] * x[p];
```

(4) 约束条件表达式。对于每种资源，所有产品消耗该资源的总量之和不能超过资源的总量。

```
subject to{
    forall (r in Resources) sum(p in Products)a[r][p] * x[p]<=b[r];
}
```

1.2.3 案例 1

Ajax 计算机公司生产计划问题。Ajax 计算机公司销售三种计算机：台式计算机 Alpha、笔记本电脑 Beta 和工作站计算机 Gamma。三种产品消耗稀有资源和资源的总量信息如

表 1-1 所示。

表 1-1 产品生产信息

产品	A-line	C-line	Labor hour	利润/元
Alpha	1		10	350
Beta	1		15	470
Gamma		1	20	610
资源总量/(小时/周)	120	48	2000	

定义数据文件内容如下：

Products= {"Alphas", "Betas", "Gammas"};

Resources= {"A-line", "C-line", "LaborHour"};

b=[120, 48, 2000];

a=[[1,1,0], [0,0,1], [10,15,20]];

c=[350, 470, 610];

运行线性规划的 OPL 模型，得到模型最优解 $x = [120 \ 0 \ 40]$ ，即生产 Alpha120 单位，不生产 Beta，生产 Gamma40 单位，最大利润为 66400 元。

1.3 数据包络分析

数据包络分析(Data Envelopment Analysis, DEA)根据多项投入指标和多项产出指标,利用线性规划的方法,对具有可比性的同类型单位进行相对有效性评价的一种数量分析方法。

在 DEA 中通常称被衡量绩效的组织为决策单元(Decision Making Unit, DMU)。决策单元的数量指的是有多少并列的机构。现有 n 个机构，已知各个机构的投入和产出， x_{ij} 代表第 j 个机构第 i 项投入，用 y_{rj} 代表第 j 个机构第 r 项产出(第一个下标指的是投入或产出的指标号；第二个下标指的是机构号)。现在要衡量某个机构 j_0 是否 DEA 有效。

$$\text{投入} \left\{ \begin{matrix} \rightarrow 1 \\ \rightarrow 2 \\ \dots \\ \rightarrow m \end{matrix} \right\} \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \dots & \dots & x_{ij} & \dots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} y_{11} & y_{12} & \dots & y_{1n} \\ y_{21} & y_{22} & \dots & y_{2n} \\ \dots & \dots & y_{rj} & \dots \\ y_{s1} & y_{s2} & \dots & y_{sn} \end{bmatrix} \left\{ \begin{matrix} 1 \rightarrow \\ 2 \rightarrow \\ \dots \\ m \rightarrow \end{matrix} \right\} \text{产出}$$

1.3.1 数学模型的建立

(1) DEA 问题的数据组织，对于 DEA 问题,所给的已知数据就是 n 个机构的 m 项投入和 s 项产出，因此数据组织成二维向量。

投入向量和产出向量如下所示：

$$\text{Inputs} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \quad \text{Outputs} = \begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1n} \\ y_{21} & y_{22} & \cdots & y_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ y_{s1} & y_{s2} & \cdots & y_{sn} \end{bmatrix}$$

(2) 数学模型。

设决策变量 E ($E \geq 0$ 并且 $E \leq 1$), λ_j ($j=1, \dots, n$) 是大于或等于 0 且小于或等于 1 的数, 且 $\sum_{j=1}^n \lambda_j = 1$ 。

现假设有一个新的机构, 这个新的机构的第 i 项投入由下式表示:

$$\sum_{j=1}^n \lambda_j x_{ij} \quad (i=1, 2, \dots, m) \text{ 且 } \sum_{j=1}^n \lambda_j = 1 (\lambda_j \geq 0)$$

该新的机构的第 r 项产出由下式表示:

$$\sum_{j=1}^n \lambda_j y_{rj} \quad (r=1, 2, \dots, s) \text{ 且 } \sum_{j=1}^n \lambda_j = 1 (\lambda_j \geq 0)$$

如果这个新机构的所有产出均大于待评价的机构 j_0 , 所有投入均小于待评价机构 j_0 , 则认为待评价机构 j_0 是 DEA 有效的。

数学模型为

$$\begin{aligned} & \min E \\ & s.t. \begin{cases} \sum_{j=1}^n \lambda_j y_{ri} \geq y_{rj_0}, & r=1, \dots, s \\ \sum_{j=1}^n \lambda_j x_{ij} \leq E x_{ij_0}, & i=1, \dots, m \\ \sum_{j=1}^n \lambda_j = 1, & j=1, \dots, n \\ \lambda_j \geq 0, & j=1, \dots, n \end{cases} \end{aligned}$$

求解结果为 $E < 1$ 时, 则 j_0 非 DEA 有效, 否则 DEA 有效。

1.3.2 DEA 的 OPL 模型

(1) 数据的定义。定义投入指标数、产出指标数、总机构数和待评护的机构号, 主要用于投入和产出数组的索引值上限。

投入指标数: `int nbInputs=...`;

产出指标数: int nbOutputs=...;

总机构数: int nbDMUs=...;

待评价的机构号: int selected=...;

当 selected 分别为 1、2、3 和 4 时,分别对应于评价机构 1、机构 2、机构 3 和机构 4,依此类推。这样做,增加了模型的适应性。

(2) 定义投入矩阵和产出矩阵。

投入矩阵: float Inputs[1..nbInputs][1..nbDMUs]=...;

产出矩阵: float Outputs[1..nbOutputs][1..nbDMUs]=...;

(3) 数据初始化。在与模型关联的数据文件中,按照数学模型中的投入和产出向量的顺序输入数据。

(4) 定义决策变量 E 和参数 lambda。

dvar float+ E;

dvar float+ lambda [1..nbDMUs];

(5) 定义目标函数为 E 取得最小值。

minimize E;

(6) 定义约束条件。

subject to{

forall (i in 1.. nbInputs)

sum(j in 1.. nbDMUs)Inputs[i][j] * lambda [j]<=Inputs[i][selected] * E;

forall (i in 1.. nbOutputs)

sum(j in 1.. nbDMUs)Outputs[i][j] * lambda [j]>=Outputs[i][selected];

sum (i in 1.. nbDMUs)lamda[i]==1;

}

1.3.3 案例 2

振华银行的 4 个分理处的投入产出情况如表 1-3 所示。要求分别确定各分理处的运行是否 DEA 有效。

表 1-2 各分理处的投入产出情况

分理处	投入		产出		
	职员数/人	营业面积/m ²	储蓄存取/万元	贷款/万元	中间业务/万元

分理处 1	15	140	1800	200	1600
分理处 2	20	130	1000	350	1000
分理处 3	21	120	800	450	1300
分理处 4	20	135	900	420	1500

在这个问题中，共有 4 个机构、2 个投入指标和三个产出指标。因为要分别对每个分理处的运行验证其 DEA 是否有效，在运行模型时，把 selected 的值分别设为 1、2、3 和 4 后运行模型。现对分理处 2 进行验证。定义数据文件内容为

```

nbInputs=2;
nbOutputs=3;
nbDMUs=4;
selected =2;
Inputs= [[15, 20,21, 20],[140,130,120,135]];
Outputs=[[1800,1000,800,900],
[200,350,450,420],
[1600,1000,1300,1500]];

```

运行 DEA 的 OPL 模型，得到结果如下，结果显示 $E = 0.9661 < 1$ ，说明分理处 2 非 DEA 有效。分理处 1、分理处 3 和分理处 4 的 DEA 有效性请自行计算。

```

E = 0.9661;
lambda = [0.27966 0 0.72034 0];

```

1.4 练习

某公司面临一个是外包协作还是自行生产的问题。该公司生产甲、乙、丙 3 种产品，这 3 种产品都要经过铸造、机加工和装配 3 个车间。甲、乙两种产品的铸件可以外包协作，也可以自行生产，但产品丙必须本厂铸造才能保证质量。有关情况见表 1-3。该公司中可利用的总工时为：铸造 8000 小时、机加工 12000 小时和装配 10000 小时。为使该公司获得最大利润，甲、乙、丙 3 种产品应各生产多少件？甲、乙两种产品由本公司铸造多少件？外包协作铸造多少件？

表 1-3 公司工时与成本数据表

产品 工时与成本	甲	乙	丙
每件铸造工时（小时）	5	10	7
每件机加工工时（小时）	7	5	8

每件装配工时（小时）	3	2	2
自产铸件每件成本（元）	3	5	4
外协铸件每件成本（元）	6	7	—
机加工每件成本（元）	2	1	3
装配每件成本（元）	3	2	2
每件产品售价（元）	23	18	16

2. 运输问题

物流业成为第三利润源，虽然不能仰仗物流业本身创造价值，但能靠合理规划最大程度地减少投入，也就意味着“创造”了价值。而在整个物流成本中，运输成本和储存成本占物流成本的主要部分。因此，合理规划运输将会降低物流成本。

2.1 实验目的

- (1) 掌握运输问题的基本模型；
- (2) 掌握不平衡运输问题的求解方法。
- (3) 将不同问题转化为运输问题进行求解。
- (4) 熟练使用 OPL 语言求解运输问题。

2.2 产销平衡运输问题

2.2.1 数学模型的建立

某种物资有 m 个产地 $A_1, A_2, \dots, A_m$ ，各产地的产量为 $a_1, a_2, \dots, a_m$ ；有 n 个销地 $B_1, B_2, \dots, B_n$ ，销地的销量为 $b_1, b_2, \dots, b_n$ ；设从产地 $A_i (i=1, 2, \dots, m)$ 向销地 $B_j (j=1, 2, \dots, n)$ 运输单位物资的运价为 c_{ij} ，问如何调运这些物资才能使总运费最小。

可以用表 2-1 的运价表直观地表达上面的问题。

表 2-1 运价表

销地 产地	B ₁	B ₂	...	B _n	产量
A ₁	c_{11}	c_{12}	...	c_{1n}	a_1
A ₂	c_{21}	c_{22}	...	c_{2n}	a_2
...	...	...		...	...
A _m	c_{m1}	c_{m2}	...	c_{mn}	a_m
销量	b_1	b_2	...	b_n	

(1) 数据的组织

该问题的数据可以组织成矩阵形式。

$$c = \begin{pmatrix} c_{11}, \dots, c_{1n} \\ \vdots \\ c_{m1}, \dots, c_{mn} \end{pmatrix}$$

(2) 数学模型

设 $x_{ij} (i=1,2,\dots,m; j=1,2,\dots,n)$ 为由产地 A_i 运到销地 B_j 的运输数量。如果总产量和总销量相等，则该问题为产销平衡问题，数学模型可以表示为

$$\begin{aligned} \min z = & \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.t.} \quad & \begin{cases} \sum_{i=1}^m x_{ij} = b_j, (j=1,2,\dots,n) \\ \sum_{j=1}^n x_{ij} = a_i, (i=1,2,\dots,m) \\ x_{ij} \geq 0, (i=1,2,\dots,m; j=1,2,\dots,n) \end{cases} \end{aligned}$$

2.2.2 运输问题的 OPL 模型

对于产销平衡运输问题，数学模型形式一致，给出产地和销地之间的运价，便可以求出最有调运方案。运价数据在 OPL 模型中可以组织成二维数组。

(1) 数据的定义和初始化。定义集合变量 OCities 为产地集合，定义 DCities 为销地集合。

```
{string}OCities=...;
```

```
{string}DCities=...;
```

定义数组 Supply[OCities]存储各个产地的产量、定义数组 Demand[DCities]存储各个销地的需求量，定义二维数组 Cost[OCities][DCities]存储产地和销地之间的运价。

```
float Supply[OCities]=...;
```

```
float Demand[DCities]=...;
```

```
float Cost[OCities][DCities]=...;
```

(2) 定义决策变量的定义。定义二维数组 Trans[OCities][DCities]存放产地和销地之间的运输数量。

```
dvar float+ Trans[OCities][DCities];
```

(3) 目标函数的定义。该问题的目标函数是调运方案所产生的总运费最小。

```
minimize sum(o in OCities,d in DCities)Cost[o][d]*Trans[o][d];
```

(4) 约束条件定义。根据上面所定义的模型，分别写出两组约束条件。

约束条件 $\sum_{j=1}^n x_{ij} = a_i (i=1,2,\dots,m)$ 所对应的 OPL 表达式为：

```
forall(o in OCities)sum(d in DCities)Trans[o][d]==Supply[o];
```

约束条件 $\sum_{i=1}^m x_{ij} = b_j (j=1,2,\dots,n)$ 所对应的 OPL 表达式为:

```
forall(d in DCities)sum(o in OCities)Trans[o][d]==Demand[d];
```

合并两个约束条件如下:

```
subject to{
```

```
forall(o in OCities)sum(d in DCities)Trans[o][d]==Supply[o];
```

```
forall(d in DCities)sum(o in OCities)Trans[o][d]==Demand[d];
```

```
}
```

2.2.3 案例 1

已知某公司有 A1、A2 和 A3 三个工厂生产某种产品，分别运往四个门市部 B1、B2、B3 和 B4 销售。有关各厂的产量、各部门的销量及运价等信息如表 2-2 所示。问如何组织运输，使总运输成本最少。

表 2-2 各厂的产量、各部门的销量及运价

工厂	B1	B2	B3	B4	产量
A1	3	11	3	10	7
A2	1	9	2	8	4
A3	7	4	10	5	9
销量	3	6	5	6	20

定义数据文件的内容为

```
OCities={"A1","A2","A3"};
```

```
DCities={"B1","B2","B3","B4"};
```

```
Supply=[7,4,9];
```

```
Demand=[3,6,5,6];
```

```
Cost=[[3,11,3,10],[1,9,2,8],[7,4,10,5]];
```

运行产销平衡运输问题的 OPL 模型，得到模型的最优解，见表 2-3，最小运费为 85.

表 2-3 求解结果

工厂	B1	B2	B3	B4	产量
A1			5	2	7
A2	3			1	4
A3		6		3	9
销量	3	6	5	6	20

2.3 产销不平衡运输问题

所谓不平衡的运输问题是指总产量不等于总销量的运输问题。在实际问题中，产销量往往是不平衡的，为了利用作业法求解，就往往需要把不平衡的运输问题化成平衡的运输问题。其基本思路是引入松弛变量，相当于增加一个虚拟的产地或销地。

2.3.1 供过于求

供过于求表示总产量大于总销量，即： $\sum_{i=1}^m a_i > \sum_{j=1}^n b_j$ ，由于总产量大于总销量，某些产地的产量调运不出去，即调运量小于其产量；由此可以建立供过于求的数学模型：

$$\begin{aligned} \min Z &= \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.t.} \quad &\begin{cases} \sum_{j=1}^n x_{ij} \leq a_i & i=1, 2, \dots, m \\ \sum_{i=1}^m x_{ij} = b_j & j=1, 2, \dots, n \\ x_{ij} \geq 0, & i=1, 2, \dots, m; j=1, 2, \dots, n \end{cases} \end{aligned}$$

由于产品供大于求，应考虑把多余的物资就地贮存，做法上即增加一个虚拟销地 B_{n+1} ，虚拟销地 B_{n+1} 的总销量为： $b_{n+1} = \sum_{i=1}^m a_i - \sum_{j=1}^n b_j$ 。

令 $x_{i(n+1)}$ 是从产地 A_i 到虚拟销地 B_{n+1} 的调运量，它相当于产地 A_i 的贮存量，不需花运费，因而运价为 0： $c_{i(n+1)} = 0$ ，在这个意义下把不平衡运输问题化为了平衡运输问题。具体求解时，只在运价表右端增加一列 B_{n+1} ，运价为零，销量为 b_{n+1} 即可。

2.3.2 供不应求

供不应求表示总产量小于总销量，即： $\sum_{i=1}^m a_i < \sum_{j=1}^n b_j$ ，由于总产量小于总销量，某些销地的需求得不到满足，即调入量小于其销量；由此可以建立供不应求的数学模型。

$$\min Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

$$\text{s.t.} \begin{cases} \sum_{j=1}^n x_{ij} = a_i & i = 1, 2, \dots, m \\ \sum_{i=1}^m x_{ij} \leq b_j & j = 1, 2, \dots, n \\ x_{ij} \geq 0, & i = 1, 2, \dots, m; j = 1, 2, \dots, n \end{cases}$$

由于供不应求，则应设想一个虚拟产地 A_{m+1} ，并让虚拟产地 A_{m+1} 来供给销地 B_j 所需物资差额。虚拟产地 A_{m+1} 的产量为： $a_{m+1} = \sum_{j=1}^n b_j - \sum_{i=1}^m a_i$

类似的，令虚拟产地 A_{m+1} 到各销地的运价为 0： $c_{(m+1)j} = 0$ 。具体计算时，在运价表的下方增加一行 A_{m+1} ，运价为零。产量为 a_{m+1} 即可。

2.3.3 案例 2

饮料厂生产一种果汁饮料，由于产品与季节关系密切，其生产能力与成本的每个季节都有不同。已知饮料厂全年 4 个季度的生产能力分别为 50 万桶、64 万桶、56 万桶和 20 万桶，4 个季度的订货数量分别为 20 万桶、28 万桶、45 万桶和 35 万桶。在成本方面：一季度生产，一至四季度销售的成本分别是 8.8 万元、8.9 万元、9 万元和 9.1 万元；二季度生产，二至四季度销售的成本分别是 9.1 万元、9.2 万元和 9.3 万元；三季度生产，三至四季度销售的成本分别是 9 万元和 9.1 万元；四季度生产，四季度销售的成本是 9.4 万元。成本中包含了本季度生产其他季度销售的存储费。在完成订货供应的情况下，如何制订饮料厂全年的生产计划使总成本最低？

【问题分析】为求解这个问题，可以把问题想象成运输问题，四个季度就是四个产地，四个季度也是四个销地。各个季度生产的饮料是经过“运输”运到各个季度去销售的，运价就是销售成本。

运输环节要受到客观条件的制约，因为只考虑一年的生产计划，所以当前季度生产的饮料只能“运到”当前季度及以后的季度中销售。同时，把在“运输方案中”禁止使用的运输路线的运价设置成 M，从而阻止选择这个运输路线。

这是个产销不平衡的运输问题。因产量大于销量，因此，考虑增加虚拟销地，使其运价为 0，销量为 65 万桶。最终得到运价表 2-4。

表 2-4 运价表

季度	一季度	二季度	三季度	四季度	虚拟季度	产量
一季度	8.8	8.9	9	9.1	0	50

二季度	M	9.1	9.2	9.3	0	64
三季度	M	M	9	9.1	0	56
四季度	M	M	M	9.4	0	20
销量	20	28	45	35	62	190

在此问题中，令 M 取值为 1000，同时定义数据文件的内容为

OCities={"A1","A2","A3","A4"};

DCities={"B1","B2","B3","B4","B5"};

Supply=[50,64,56,20];

Demand=[20,28,45,35,62];

Cost=[[8.8,8.9,9,9.1,0],

[1000,9.1,9.2,9.3,0],

[1000,1000,9,9.1,0],

[1000,1000,1000,9.4,0]];

运行产销平衡运输问题的 OPL 模型，得到模型的最优解，见表 2-5，最小运费为 1153.1 万元。

表 2-5 求解结果

季度	一季度	二季度	三季度	四季度	虚拟季度	产量
一季度	20	6		24		50
二季度		22			42	64
三季度			45	11		56
四季度					20	20
销量	20	28	45	35	62	190

2.4 练习

设有三个煤矿供应四个电厂的发电用煤。假定各个煤矿的年产量、各个电厂的备用煤量以及单位运价如表 2-6 所示。试求运费最少的煤炭调拨方案。

表 2-6 备用煤量以及单位运价

煤矿	I	II	III	IV	产量
A	16	13	22	17	55
B	14	13	19	15	55
C	19	20	23	-	50
最低需要量	30	70	0	10	

最高需求量	50	70	30	不限	
-------	----	----	----	----	--

3. 整数规划

3.1 实验目的

- (1) 理解整数规划的概念。
- (2) 对于整数规划的问题，能够建立基本的整数规划模型，
- (3) 会运用 OPL 优化建模语言解决 0-1 整数规划问题。

3.2 0-1 整数规划模型

规划问题中一部分或全部决策变量必须取非负整数值的规划问题称为整数规划。不考虑整数条件，规划问题如果是线性规划，则该整数规划称为整数线性规划。整数线性规划分为以下几种类型。

- (1) 纯整数线性规划：指全部决策变量都必须取非负整数值的整数线性规划。
- (2) 混合整数线性规划：指决策变量中的一部分必须取非负整数值，另一部分可以不取整数值的整数线性规划。
- (3) 0-1 整数线性规划：指决策变量只能取值 0 或 1 的整数线性规划。

纯整数线性规划和混合整数线性规划在建立 OPL 模型上并不存在特殊性，因为只要把那些有整数要求的决策变量定义成整型决策变量就可以了。比较特殊的是 0-1 整数线性规划。实际应用中，使用最多的是 0-1 整数规划，因此此处只讲解 0-1 规划的软件求解方法。

IBM ILOG OPL 语言可以用于定义决策变量的类型包括整型、浮点型和布尔型。布尔型决策变量限定决策变量的取值或者为 0，或者为 1。因此，在 0-1 整数线性规划 OPL 模型的创建上，把取值限定在 0 和 1 的范围内的决策变量定义为布尔型决策变量。0-1 整数线性规划的数学模型为

$$\begin{aligned} & \max(\text{or min}) \sum_{j=1}^n c_j x_j \\ & \text{s.t.} \begin{cases} \sum_{j=1}^n a_{ij} x_j \leq b_i, (i=1, 2, \dots, m) \\ x_j \in \{0, 1\}, (j=1, 2, \dots, m) \end{cases} \end{aligned}$$

3.3 仓库选址问题

仓库选址是一个典型的离散优化问题。公司考虑在一些地点设计仓库以供应若干商店。每个可能的仓库都有固定成本，能够供应有限个商店。每个商店只能由一个仓库供应。各个仓库供应各个商店的成本不同。要求确定在哪些地点建立仓库才能使总成本最

低。

【问题分析】仓库选址问题一般会给出仓库的数量、商店的数量、仓库供应商店的成本等信息，有时候还给出设立仓库的固定成本。要求确定的是在哪些地点建立仓库，每个仓库供应哪些商店，从而使供应成本最低。仓库选址问题的约束条件一般会包括：

- (1) 每个商店都必须有仓库供应，并且每个商店只能由一个仓库供应；
- (2) 每个仓库供应的商店数量有限；
- (3) 每个商店只能由建立的仓库供应。

3.3.1 数学模型的建立

(1) 数据的组织。假设有 m 个商店、 n 个仓库，把商店由各个仓库的供应成本组织成二维数组 c ，把仓库的固定成本组织成一维数组 f 。

(2) 数学模型。设：

$$x_{ij} = \begin{cases} 1 & \text{第}i\text{个商店由第}j\text{个仓库供应} \\ 0 & \text{第}i\text{个商店不由第}j\text{个仓库供应} \end{cases}$$

$$y_j = \begin{cases} 1 & \text{建立第}j\text{个仓库} \\ 0 & \text{不建立第}j\text{个仓库} \end{cases}$$

$$\begin{aligned} \min & \sum_{i=1}^m \sum_{j=1}^n x_{ij} c_{ij} + \sum_{j=1}^n y_j f_j \\ \text{s.t.} & \begin{cases} \sum_{j=1}^n x_{ij} = 1 & (i=1,2,\dots,m) \\ \sum_{i=1}^m x_{ij} \leq b_j & (j=1,2,\dots,n) \\ x_{ij} \leq y_j & (i=1,2,\dots,m, j=1,2,\dots,n) \\ x_{ij}, y_j \in \{0,1\} & (i=1,2,\dots,m, j=1,2,\dots,n) \end{cases} \end{aligned}$$

3.3.2 仓库选址问题的 OPL 模型

(1) 数据结构的定义及数据的初始化。

```
{string}} Warehouses=...;
int Fixed=...;
int NbStores=...;
range Stores=1..NbStores;
int Capacity[Warehouses]=...;
int SupplyCost[Stores][Warehouses]=...;
```

(2) 决策变量的定义。定义一维数组 $Open[j]$ 表示仓库 j 是否建立，定义二维数组 $Supply[Stores][Warehouses]$ 表示仓库 j 是否向商店 i 供货；

```
dvar boolean Open[Warehouses];
```

```
dvar boolean Supply[Stores][Warehouses];
```

(3) 目标函数的定义。

```
minimize sum(w in Warehouses, s in stores)
```

```
SupplyCost [s][w] * Supply[s][w] + sum( w in Warehouses) Fixed*Open[w];
```

(4) 约束条件的定义。

```
subject to{
```

```
forall(s in Stores)sum(w in Warehouses)Supply[s][w]==1;
```

```
forall(w in Warehouses)sum(s in Stores)Supply[s][w]<=Capacity[w];
```

```
forall(w in Warehouses)
```

```
    forall(s in Stores)
```

```
        Supply[s][w]<=Open[w];
```

```
}
```

3.3.3 案例 1

公司考虑在 5 个候选地点设计若干仓库以供应 10 个商店，每个可能的仓库都有固定成本 30，能够供应有限个商店。每个商店只能由一个仓库供应。各个仓库供应各个商店的成本不同，如表 3-1 所示。要求确定在哪些地点建立仓库才能使总成本最低。

表 3-1 各个仓库供应各个商店的成本

仓库	仓库 1 Bonn	仓库 2 Bordeaux	仓库 3 London	仓库 4 Paris	仓库 5 Rome
仓库供应能力/个	1	4	2	1	3
商店 1	20	24	11	25	30
商店 2	28	27	82	83	74
商店 3	74	97	71	96	70
商店 4	2	55	73	69	61
商店 5	46	96	59	83	4
商店 6	42	22	29	67	59
商店 7	1	5	73	59	56
商店 8	10	73	13	43	96
商店 9	93	35	63	85	46
商店 10	47	65	55	71	95

定义数据文件内容如下：

Warehouses={Bonn,Bordeaux,London,Paris,Rome};

Fixed=30;

NbStores=10;

Capacity = [1,4,2,1,3];

SupplyCost=[
[20,24,11,25,30],[28,27,82,83,74],
[74,97,71,96,70],[2,55,73,69,61],
[46,96,59,83,4],[42,22,29,67,59],
[1,5,73,59,56],[10,73,13,43,96],
[93,35,63,85,46],[47,65,55,71,95]];

运行仓库选址问题的 OPL 模型，求解结果如下，最小费用为 383。建立仓库 1、仓库 2、仓库 3 和仓库 5，供货方案为仓库 1 向商店 4 供货，仓库 2 向商店 2、6、7、9 供货，仓库 3 向商店 8、10 供货，仓库 5 向商店 1、3、5 供货。

Supply = [
[0 0 0 0 1]
[0 1 0 0 0]
[0 0 0 0 1]
[1 0 0 0 0]
[0 0 0 0 1]
[0 1 0 0 0]
[0 1 0 0 0]
[0 0 1 0 0]
[0 1 0 0 0]
[0 0 1 0 0]];
Open = [1 1 1 0 1];

3.4 指派问题

在现实生活中经常遇到给员工安排任务的情况。某单位有 n 件事要完成，有 m 个员工可以负责这 n 件事。由于每个人的专长不同，每个人完成这 n 件事的效率(成本)也不同。如何安排哪个人完成哪件事，使完成所有事的总效率最高(总成本最低)。这类问题被称为指派问题。

假设人数和事数相等，已知第 i 人做第 j 事的费用 c_{ij} ，要求确定人和事之间的一一对应的指派方案，使完成这 n 件事的总费用最少。

3.4.1 数学模型的建立

(1) 数据的组织。把第 i 个人做第 j 件事的费用组织成二维向量 c_{ij} 。

(2) 数学模型。设 x_{ij} 表示是否派第 i 个人做第 j 件事，如果派第 i 个人做第 j 件事，则 $x_{ij}=1$ ；否则 $x_{ij}=0$ 。

$$\begin{aligned} \min z &= \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ s.t. &\begin{cases} \sum_{i=1}^n x_{ij} = 1 & (j=1, 2, \dots, n) \\ \sum_{j=1}^n x_{ij} = 1 & (i=1, 2, \dots, n) \\ x_{ij} \in \{0, 1\} & (i=1, 2, \dots, n, j=1, 2, \dots, n) \end{cases} \end{aligned}$$

3.4.2 指派问题的 OPL 模型

(1) 数据的定义。把人员和事情的名称组织成集合，做数组的下标和形参的值域。

```
{string}persons=...;
```

```
{string}clients=...;
```

```
float cost[persons][clients]=...;
```

(2) 决策变量的定义。

定义布尔型二维数组 $x[\text{persons}][\text{clients}]$ 的值表示人员 i 和完成工作 j 。如果人员 i 和完成工作 j ，则 $x[i][j]=1$ ；否则 $x[i][j]=0$ 。

```
dvar boolean x[persons][clients];
```

(3) 目标函数的定义。

```
minimize sum(i in persons, j in clients) cost[i][j]*x[i][j];
```

(4) 约束条件的定义。

```
subject to{
```

```
forall(i in persons)sum(j in clients) x[i][j]==1;
```

```
forall(j in clients)sum(i in persons) x[i][j]==1;
```

```
}
```


3.4.3 案例 2

福尔市场研究公司刚刚从 3 个新客户那里得到进行市场研究的要求。该公司面临的问题是如何给每个客户指派项目主管(代理商)。目前,有 3 个人手头没有其他的工作,可以胜任项目主管。然而,福尔的管理层意识到完成每项市场研究所需要的时间完全取决于该项目主管的经验和能力,具体数据如表 3-2 所示。这 3 个项目的优先顺序大致相当。公司希望其委派的项目主管都能够用最短的时间完成这 3 个项目。如果每个项目主管只指派到一个客户,那么应该怎样指派呢?

表 3-2 福尔市场营销调查配置问题的预计项目完成时间

项目主管	客 户		
	A	B	C
Terry	10	15	9
Karl	9	18	5
Bob	6	14	3

定义数据文件如下。

```
persons={"Terry", "Karl", "Bob"};
```

```
clients={"A", "B", "C"};
```

```
cost=[[10 15 9],
```

```
[9 18 5],
```

```
[6 14 3]];
```

运行指派问题的 OPL 模型,得到结果如下,最少费用为 26。安排方案为 Terry 委派客户 B, Karl 委派客户 C, Bob 委派客户 A。

```
x = [[0 1 0]
```

```
[0 0 1]
```

```
[1 0 0]];
```

3.5 练习

【固定费用问题】有三种资源被用于生产三种产品。资源量、产品单件可变费用及售价、资源单耗量及组织三种产品生产的固定费用见表 3-3。要求制定一个生产计划,使总收益最大。

表 3-3 相关费用

资源 \ 产品	I	II	III	资源量
A	2	4	8	600
B	2	3	4	400
C	1	2	3	250

单件可变费用	5	4	6	
固定费用	100	150	200	
单件售价	8	10	14	

4. 图与网络分析

4.1 实验目的

- (1) 掌握最短路问题、最大流问题和最小生成树问题的基本模型；
- (2) 熟练使用 OPL 语言求解最短路问题、最大流问题和最小生成树问题。
- (3) 学会建立最短路问题、最大流问题和最小生成树问题的模型。

4.2 最短路问题

4.2.1 数学模型的建立

已知一个网络图 $G(v, e, w)$ 如图 4-1 所示(v 是顶点集合, e 是边的集合, w 是网络图的权矩阵), 找出某一对顶点之间的最短路径。也就是说在两个点之间的所有路径中, 找到路径上的边的权值之和最小的路径。网络图可以是有向的, 也可以是无向的。通常我们把无向网络图看成是每两个顶点之间有双向边的有向网络图。

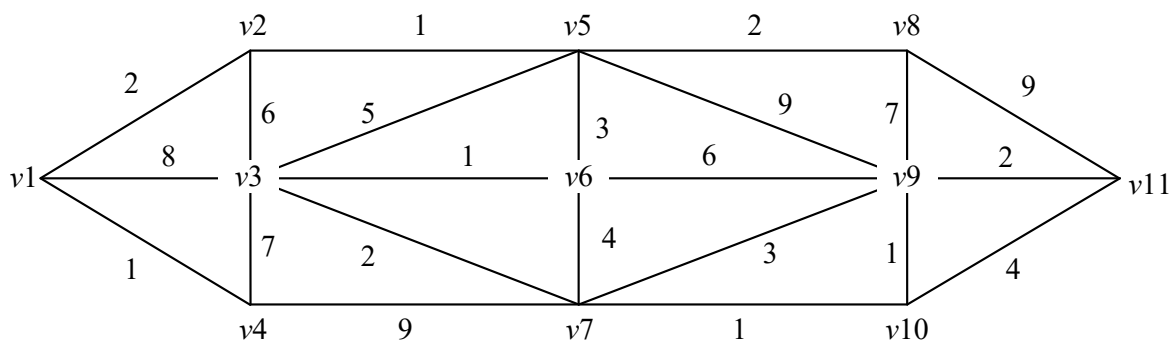


图 4-1 网络图

比如, 从 $v1$ 到 $v11$ 存在很多路径, $(v1, v2, v5, v8, v11)$ 是一条路径, $(v1, v3, v5, v8, v11)$ 也是一条路径, 还有很多这样的路径。最短路径是把各个路径上边的权值相加, 其中和最小的路径。

设 x_{ij} 为对应于网络图的边 $\langle i, j \rangle$ 在不在最短路径上的决策: 当顶点之间的路径上含边 $\langle i, j \rangle$ 时, $x_{ij}=1$; 否则 $x_{ij}=0$ 。

在 $x_{ij}=1$ ($\langle i, j \rangle$ 属于 E) 的所有组合中, 找到一个组合, 使 $x_{ij}=1$ 的 $\langle i, j \rangle$ 边的权值之和最小, 用表达式表示则为 $\sum w_{ij}x_{ij}$ 最小的组合。如果没有任何约束, 那么所有的边 $\langle i, j \rangle$ 属于 E , 对应的 $x_{ij}=1$, $\sum w_{ij}x_{ij}$ 权值之和最小为 0, 因为没有边的 $x_{ij}=1$, 也就意味着没有边在最短路径中。因此, 需要添加约束条件。

需要有若干条边的 x_{ij} 等于 1。 $x_{ij}=1$ 的所有边 $\langle i, j \rangle$ 应该构成一条从始点到终点的路

径。这条路径不一定包含所有的结点，但至少包含始点和终点。不论结点在不在最短路径上，对于每个结点，都有如下关系成立。

(1) 对于始点 1: ① $x_{1k}=1$ (k 是其中一个结点)，以 1 为起点的弧中，只能有一条弧出现在最短路径上。② $x_{j1}=1$ ($j=1,2,\dots,n$)；以 1 为终点的弧中，不能有弧出现在最短路径上，否则就构成了回路。

(2) 对于终点 n : ① $x_{kn}=1$ (k 是其中一个结点)，以 n 为终点的弧中，只能有一条弧出现在最短路径上。② $x_{nj}=1$ ($j=1,2,\dots,n$)；以 n 为起点的弧中，不能有弧出现在最短路径上，否则就构成了回路。

(3) 除了始点和终点之外的中间点 i 所在的所有边中，或者所有边都不出现在最短路径上，即所有边所对应的 x_{ij} 和 $x_{ji}=0$ ($j=1,2,\dots,n$)；或者有以 i 为起点和终点的边，各有一个出现在最短路径上，即 $x_{ik}=1$ 和 $x_{ji}=1$ (k 和 j 是其中的两个结点)。

通过总结，在有向图的最短路径上的顶点和弧有如下特点：

(1) 始点出现在起点的次数与始点出现在终点的次数差为 1。 -

(2) 终点出现在终点的次数与终点出现在起点的次数差为 1。

(3) 其他任意点出现在起点和终点的次数相同(包括从未出现在起点和终点的情况)。这些看似松散的约束条件与极小化目标函数(最短路径上弧的权值之和最小)配合，能够实现找到最短路径的目标。

其数学模型为

$$\begin{aligned} \min & \sum_{\langle i,j \rangle \in E} w_{ij} x_{ij} \\ \text{s.t.} & \begin{cases} \sum_{j \in V, \langle i,j \rangle \in E} x_{ij} - \sum_{j \in V, \langle j,i \rangle \in E} x_{ji} = \begin{cases} 1, (i=1) \\ -1, (i=n) \\ 0, (i \neq 1, n) \end{cases} \\ x_{ij} = \{0,1\}, i \in V, j \in V, \langle i,j \rangle \in E \end{cases} \end{aligned}$$

4.2.2 最短路径问题的 OPL 模型

(1) 数据的定义和初始化。定义变量 n 存储顶点个数。定义区间变量 v 表示邻接矩阵下标。定义二维数组 $\text{adj}[v][v]$ 存储邻接矩阵。定义二维数组 $w[v][v]$ 存储权值矩阵。

`int n=...`;

`range v=1..n;`

`int adj[v][v]=...`;

```
int w[v][v]=...;
```

(2) 决策变量的定义。定义布尔型二维数组 $x[v][v]$ 的值表示顶点 i 和顶点 j 在最短路径上。如果顶点 i 和 j 在最短路径上，则 $x[i][j]=1$ ；否则 $x[i][j]=0$ 。

```
dvar boolean x[v][v];
```

(3) 目标函数的定义。在最短路径上的边的权值之和最小。

```
minimize sum(i in v,j in v)x[i][j]*w[i][j];
```

(4) 约束条件的定义。

约束条件 $\sum_{j \in V, \langle i, j \rangle \in E} x_{ij} - \sum_{j \in V, \langle i, j \rangle \in E} x_{ji} = 1, i=1$ 所对应的 OPL 表达式为：

```
sum(j in v)adj[j][1]*x[j][1]+1==sum(k in v)adj[1][k]*x[1][k];
```

约束条件 $\sum_{j \in V, \langle i, j \rangle \in E} x_{ij} - \sum_{j \in V, \langle i, j \rangle \in E} x_{ji} = -1, i=n$ 所对应的 OPL 表达式为：

```
sum(j in v)adj[j][11]*x[j][11]-1==sum(k in v)adj[11][k]*x[11][k];
```

约束条件 $\sum_{j \in V, \langle i, j \rangle \in E} x_{ij} - \sum_{j \in V, \langle i, j \rangle \in E} x_{ji} = 0, i \neq 1, n$ 表示除始点和终点外，任何一个中间点的

所有的边中，选中的以该点为起点的边数和等于以该点为终点的边数和，对应的 OPL 表达式为：

```
forall(i in v:i!=1&&i!=n)sum(j in v)adj[j][i]*x[j][i]==sum(k in v)adj[i][k]*x[i][k];
```

合并约束条件如下：

```
subject to{
```

```
sum(j in v)adj[j][1]*x[j][1]+1==sum(k in v)adj[1][k]*x[1][k];
```

```
sum(j in v)adj[j][11]*x[j][11]-1==sum(k in v)adj[11][k]*x[11][k];
```

```
forall(i in v:i!=1&&i!=n)sum(j in v)adj[j][i]*x[j][i]==sum(k in v)adj[i][k]*x[i][k];
```

```
}
```

4.2.3 案例 1

求图 4-1 的无向图中从 v_1 到 v_{11} 之间的最短路径。

定义数据文件内容如下

```
adj=[[0 1 1 1 0 0 0 0 0 0 0]
```

```
[1 0 1 0 1 0 0 0 0 0 0]
```

```
[1 1 0 1 1 1 1 0 0 0 0]
```

```
[1 0 1 0 0 0 1 0 0 0 0]
[0 1 1 0 0 1 0 1 1 0 0]
[0 0 1 0 1 0 1 0 1 0 0]
[0 0 1 1 0 1 0 0 1 1 0]
[0 0 0 0 1 0 0 0 1 0 1]
[0 0 0 0 1 1 1 1 0 1 1]
[0 0 0 0 0 0 1 0 1 0 1]
[0 0 0 0 0 0 0 1 1 1 0]];
w=[[0 2 8 1 0 0 0 0 0 0 0]
[2 0 6 0 1 0 0 0 0 0 0]
[8 6 0 7 5 1 2 0 0 0 0]
[1 0 7 0 0 0 9 0 0 0 0]
[0 1 5 0 0 3 0 2 9 0 0]
[0 0 1 0 3 0 4 0 6 0 0]
[0 0 2 9 0 4 0 0 3 1 0]
[0 0 0 0 2 0 0 0 7 0 9]
[0 0 0 0 9 6 3 7 0 1 2]
[0 0 0 0 0 0 1 0 1 0 4]
[0 0 0 0 0 0 0 9 2 4 0]];
```

运行最短路问题的 OPL 模型，得到求解结果如下，最短距离为 13.

```
x = [[0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 1]
[0 0 0 0 0 0 0 0 1 0 0]
```

[0 0 0 0 0 0 0 0 0 0];

最短路径为 $v_1 \rightarrow v_2 \rightarrow v_5 \rightarrow v_6 \rightarrow v_3 \rightarrow v_7 \rightarrow v_{10} \rightarrow v_9 \rightarrow v_{11}$ ，见图 4-2.

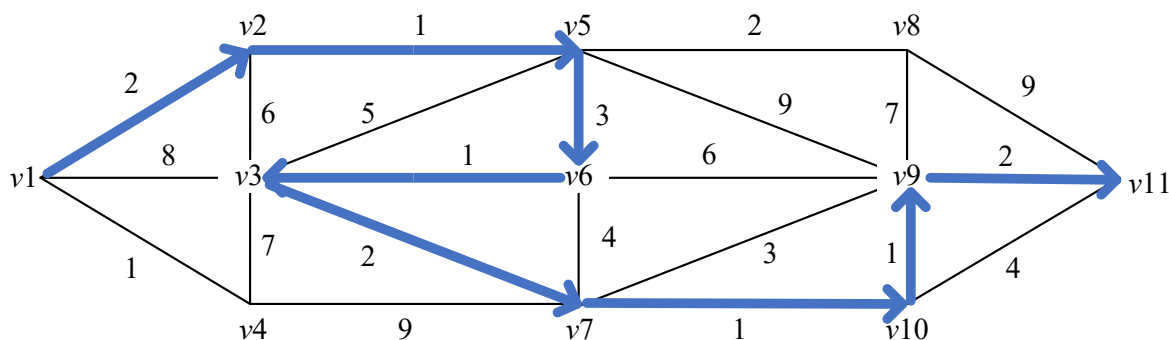


图 4-2 最短路径

4.3 最大流问题

最大流问题是一类应用极为广泛的问题，例如在交通运输网络中有人流、车流、货物流，供水网络中有水流，金融系统中有现金流，通信系统中有信息流，等等。20 世纪 50 年代，福特（Ford）、富克逊（Fulkerson）建立的“网络流理论”是网络应用的重要组成部分。

4.3.1 数学模型的建立

设一个赋权有向图 $D=(V,E)$ ，在 V 中指定一个发点 v_s 和一个收点 v_t ，其他的点叫做中间点。对于 D 中的每一个弧 $(v_i, v_j) \in E$ ，都有一个非负数 c_{ij} ，叫做弧的容量。我们把这样的图 D 叫做一个容量网络，简称网络。记作 $D=(V, E, C)$ ，如图 4-3 所示。

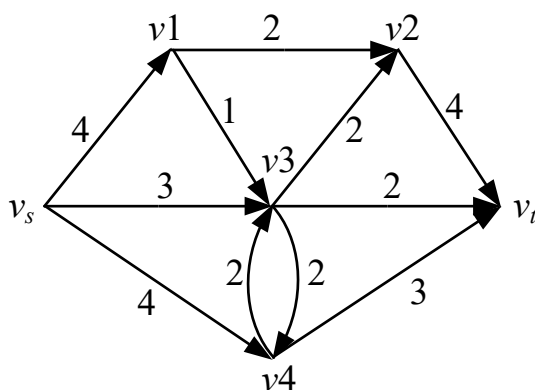


图 4-3 网络图

网络 D 上的流是指定义在弧集合 E 上的一个函数，其中 $f(v_i, v_j) = f_{ij}$ ，叫做弧 (v_i, v_j) 上的流量。满足下列条件的流称为可行流

(1) 容量条件：对于每一个弧 $(v_i, v_j) \in E$ ，有 $0 \leq f_{ij} \leq c_{ij}$ 。

(2) 平衡条件：对于发点 v_s ，有 $\sum_{(v_s, v_j) \in E} f_{sj} - \sum_{(v_j, v_s) \in E} f_{js} = -v$

对于收点 v_t ，有 $\sum_{(v_j, v_t) \in E} f_{jt} - \sum_{(v_t, v_j) \in E} f_{jt} = v$

对于中间点，有 $\sum_{(v_i, v_j) \in E} f_{ij} - \sum_{(v_i, v_j) \in E} f_{ji} = 0$

可行流中 $f_{ij} = c_{ij}$ 的弧叫做饱和弧， $f_{ij} < c_{ij}$ 的弧叫做非饱和弧。 $f_{ij} > 0$ 的弧叫做非零流弧， $f_{ij} = 0$ 的弧叫做零流弧。

因此，最大流问题的数学模型为

$$\begin{aligned} \max v \\ \sum_{(i,j) \in E} x_{ij} - \sum_{(i,j) \in E} x_{ji} &= \begin{cases} v, i = s \\ -v, i = t \\ 0, i \neq s, t \end{cases} \\ 0 \leq x_{ij} \leq c_{ij}, \forall (i, j) \in E \end{aligned}$$

4.3.2 最大流问题的 OPL 模型

(1) 数据的定义和初始化。首先要确定初始化网络图的节点数和弧的容量。定义字符串集合 `nodes`，存储顶点名称。定义结构体 `edge`，表示有向弧。定义 `edge` 类型的集合 `edges`，存储网络图中的有向弧。定义数组 `cap[edges]`，表示各个有向弧的容量。

```
{string}nodes=...;
tuple edge{
string origin;
string destination;
}
setof(edge) edges=...;
int cap[edges]=...;
```

(2) 决策变量的定义。

定义整型一维数组 `flow[edges]`，表示各个有向弧的流量。

```
dvar int+ flow[edges];
```

定义决策表达式 `maxflow`，表示始点流出的流量之和。

```
dexpr int maxflow=sum(i in edges:i.origin=="begin") flow[i];
```

(3) 目标函数的定义。定义目标函数是整个网络中的流量最大。

```
maximize maxflow;
```

(4) 约束条件的定义

对于中间点来说，流出的流量和流入的流量平衡，约束如下

```
forall(i in nodes:i!="begin"&&i!="end")
```

```
    sum(j in edges:j.origin==i)flow[j]==sum(j in edges:j.destination==i)flow[j];
```

每条弧上的流量不超过其容量，约束如下

```
forall(i in edges)flow[i]<=cap[i];
```

合并约束条件如下：

```
subject to{
```

```
forall(i in nodes:i!="begin"&&i!="end")
```

```
    sum(j in edges:j.origin==i)flow[j]==sum(j in edges:j.destination==i)flow[j];
```

```
forall(i in edges)flow[i]<=cap[i];
```

4.3.3 案例 2

求图 4-3 的网络图中的最大流。

定义数据文件内容如下

```
nodes={begin,one,two,three,four,end};
```

```
edges={<begin,one>,<begin,three>,<begin,four>,<one,two>,<one,three>,<two,end>,<three,two>,<three,four>,<three,end>,<four,three>,<four,end>};
```

```
cap=[4,3,4,2,1,4,2,2,3,2,3];
```

运行最大流问题的 OPL 模型，得到结果如下，最大流量为 9，见过见图 4-4.

```
flow = [3 3 3 2 1 4 2 0 2 0 3];
```

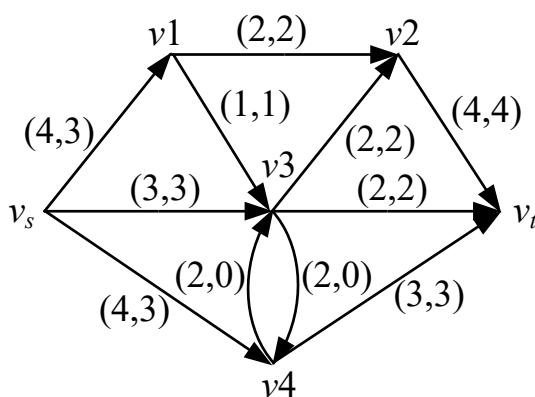


图 4-4 最大流

4.4 练习

在图 4-5 所示道路网中，要求沿道路铺设光纤网使点 v1 与其他各点可以进行通讯，求如何使得光纤的总长最小。

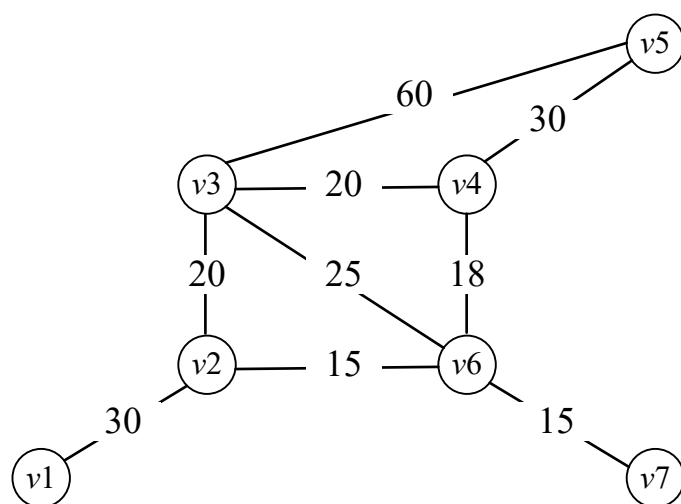


图 4-5 网络图